

2.4 Konstruktionsprinzipien von Blockverschlüsselungsverfahren

„Schleifen“ von elementaren Operationen auf den Blöcken
 l (Bits)

- Transposition von Zeichen in den Blöcken
- Substitution auf Unterblöcken (z. B. Bytes)

2.5 DES und Triple-DES ([FIPS PUB 46-3](#))

DES

Data Encryption Standard

Sowohl die Blöcke als auch die Schlüssel des DES-Verfahrens setzen sich jeweils aus 8 Bytes zusammen:

$$\mathcal{M} = (\mathbb{Z}_2^8)^8 = \{(m_1, m_2, \dots, m_8) \mid m_i \in \mathbb{Z}_2^8\} \quad 64 \text{ Bit}$$

$$\mathcal{K} = \{(k_1, k_2, \dots, k_8) \mid k_i \in \mathbb{Z}_2^8, \text{parity}(k_i) = 1\} \subseteq (\mathbb{Z}_2^8)^8 \quad 56 \text{ Bit}$$

Hierbei ist für $k_i \in \mathbb{Z}_2^8$ die *Parität* (*parity*) definiert durch:

$$\text{parity} : \mathbb{Z}_2^8 \rightarrow \mathbb{Z}_2, \quad \text{parity} \left(\sum_{i=0}^7 b_i \cdot 2^i \right) := \left(\sum_{i=0}^7 b_i \right) \bmod 2$$

Für jedes Schlüsselbyte muss die Anzahl der 1-Bits in der Binärdarstellung des Bytes also ungerade sein.

$$|\mathcal{K}| = 2^{56}$$

viel zu klein!

Triple-DES (3DES)

Die Blöcke des 3DES-Verfahrens setzen sich wie beim DES-Verfahren aus 8 Bytes zusammen:

$$\mathcal{M} = (\mathbb{Z}_2^8)^8 = \{(m_1, m_2, \dots, m_8) \mid m_i \in \mathbb{Z}_2^8\}$$

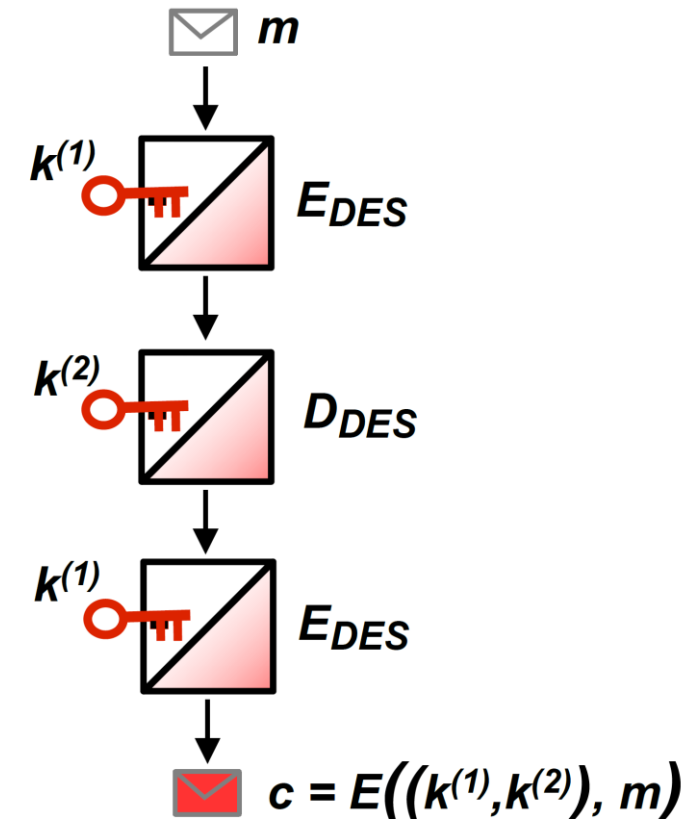
Der Schlüsselraum des 3DES-Verfahrens besteht aus der Menge der Paare von DES-Schlüsseln:

$$\mathcal{K} = \{(k_1, k_2, \dots, k_8) \mid k_i \in \mathbb{Z}_2^8, \text{parity}(k_i) = 1\}^2$$

Bezeichnen E_{DES} und D_{DES} die DES-Ver- beziehungsweise DES-Entschlüsselungsfunktion, so ist das 3DES-Verschlüsselungsverfahren definiert durch:

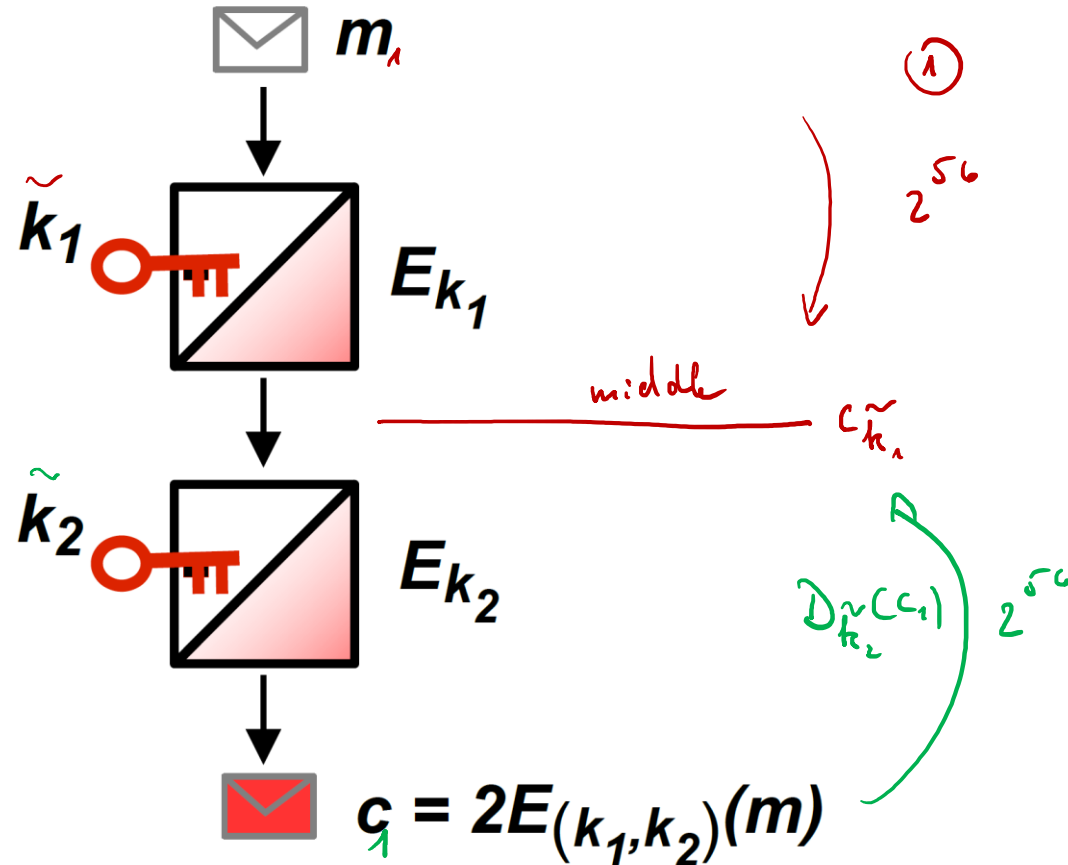
$$E\left(\left(k^{(1)}, k^{(2)}\right), m\right) := E_{DES}\left(k^{(1)}, D_{DES}\left(k^{(2)}, E_{DES}\left(k^{(1)}, m\right)\right)\right) \quad (2.2)$$

Die Hintereinanderausführung der DES-Verfahren ist in Abbildung 2.14 dargestellt.



2.6 Meet-in-the-Middle-Angriffe

Annahme: Einige Paare $(m_1, c_1), (m_2, c_2), \dots$ sind bekannt (known-Plaintext)



Konstruktion des Verschlüsselungsverfahrens $2E$

$\tilde{k}_1$	$c_{\tilde{k}_1} = E(\tilde{k}_1, m_1)$
0101010101010101	8CA64DE9C1B123A7
0101010101010102	2F3F20C13DEF464C
0101010101010104	6CB9F25ABAAA9DDD
0101010101010107	A28C9CC39D2A5B8D
$\vdots$	$\vdots$
FEFEFEFEFEFEFEFE8	92A838AD92ADA020
FEFEFEFEFEFEFEFEFB	8D8CA49AF5475AB8
FEFEFEFEFEFEFEFEFD	614650AFAE8FC8C0
FEFEFEFEFEFEFEFEFE	CAAAAF4DEAF1DBAE

(2) Tabelle hat
2-te Spalte sortieren
 $\sim 56 \cdot 2^{56}$

AES-128

Spezifiziert ist AES für Schlüssel der Bitlängen 128, 192 und 256. Der grundsätzliche Aufbau der entsprechenden Algorithmen AES-128, AES-192 und AES-256 ist jedoch gleich. Die folgende Beschreibung beschränkt sich daher auf eine Betrachtung von AES-128 und die Bezeichnung AES (ohne weitere Nennung der Schlüssellänge) wird synonym für AES-128 benutzt.

Sowohl die Blöcke als auch die Schlüssel setzen sich jeweils aus 16 Bytes zusammen:

$$\mathcal{M} = (\mathbb{Z}_2^8)^{16} = \{(m_1, m_2, \dots, m_{16}) \mid m_i \in \mathbb{Z}_2^8\}$$

$$\mathcal{K} = (\mathbb{Z}_2^8)^{16} = \{(k_1, k_2, \dots, k_{16}) \mid k_i \in \mathbb{Z}_2^8\}$$

Blöcke werden zur Beschreibung des AES-Algorithmus mit den (4,4)-Matrizen mit Einträgen aus $\mathcal{Z} = \mathbb{Z}_2^8$ identifiziert und in dieser Form auch als *State-Array* bezeichnet. Ein Eingabeblock $(m_1, m_2, \dots, m_{16})$ wird hierbei zunächst wie folgt geschrieben:

$$\mathbf{S} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} := \begin{pmatrix} m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \\ m_4 & m_8 & m_{12} & m_{16} \end{pmatrix}$$

$$|\mathcal{M}| = 2^{128}$$

$$|\mathcal{K}| = 2^{128}, 2^{192}, 2^{256}$$

Im AES sind vier elementaren Operationen auf den als State-Arrays dargestellten Blöcken definiert:

(1) SubBytes()

SubBytes() führt die in Abbildung 2.17 definierte Substitution S auf allen Einträgen des State-Arrays aus:

$$\begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} \mapsto \begin{pmatrix} S(s_{0,0}) & S(s_{0,1}) & S(s_{0,2}) & S(s_{0,3}) \\ S(s_{1,0}) & S(s_{1,1}) & S(s_{1,2}) & S(s_{1,3}) \\ S(s_{2,0}) & S(s_{2,1}) & S(s_{2,2}) & S(s_{2,3}) \\ S(s_{3,0}) & S(s_{3,1}) & S(s_{3,2}) & S(s_{3,3}) \end{pmatrix}$$

	*0	*1	*2	*3	*4	*5	*6	*7	*8	*9	*A	*B	*C	*D	*E	*F
0*	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1*	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2*	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3*	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4*	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5*	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6*	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7*	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8*	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9*	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A*	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B*	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C*	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D*	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E*	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F*	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Bsp.

$$S(\underline{42}) = 0 \times 2C$$

S besitzt algebraische "kurze" Abbildungsvorschrift $\rightarrow$ ① + Implement. in HW
② + Kryptanalyse

(2) ShiftRows()

ShiftRows() führt eine Transposition aus, die folgendermaßen auf Ebene der State-Array-Einträge definiert ist:

$$\begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} \mapsto \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{pmatrix}$$

Transposition

(3) MixColumns()

MixColumns() führt eine Substitution auf den Spalten des State-Arrays $\mathbf{S}$ aus, die durch eine Multiplikation mit einer fest definierten Matrix $\mathbf{M}$ definiert ist:

$$\mathbf{S} \mapsto \mathbf{M} \cdot \mathbf{S}, \quad \text{mit } \mathbf{M} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$$

Substitution
(auf Spalten)

Zur Berechnung der Einträge in $\mathbf{M} \cdot \mathbf{S}$ werden Summen und Produkte von Bytewerten **nicht** in $\mathbb{Z}_{256}$ berechnet. (Andernfalls definierte die Matrizenmultiplikation auch keine injektive Abbildung und eine Verschlüsselung wäre dann nicht mehr eindeutig umkehrbar!) Zur Berechnung der Einträge in $\mathbf{M} \cdot \mathbf{S}$ sind die Bytewerte vielmehr als Elemente des Körpers $\mathbb{F}_{256}$ aufzufassen, der bezüglich des irreduziblen Polynoms

$$m(x) := x^8 + x^4 + x^3 + x + 1 \in \mathbb{F}_2[x] \quad (2.8)$$

definiert ist.

(4) AddRoundKey()

Nach einer initialen Schlüsselexpansion, die hier nicht weiter besprochen werden soll, steht für jede Runde ein aus 16 Bytes zusammengesetzter Rundenschlüssel zur Verfügung, der jeweils als Matrix

$$\mathbf{K} = \begin{pmatrix} k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \end{pmatrix}$$

dargestellt werden kann.

AddRoundKey() führt einfach eine XOR-Operation aller Einträge des State-Arrays $\mathbf{S}$ mit den entsprechenden Einträgen aus $\mathbf{K}$ durch. In $\mathbb{F}_{256}$ entspricht dies einer Addition der Einträge und damit der Matrizenaddition:

$$\mathbf{S} \mapsto \mathbf{K} + \mathbf{S}$$

```

AES_Encrypt(S, K0, K1, ..., K10) {
    AddRoundKey(S, K0);
    for( int i=1; i < 10; ++i ){
        SubBytes(S);
        ShiftRows(S);
        MixColumns(S);
        AddRoundKey(S, Ki);
    }
    SubBytes(S);
    ShiftRows(S);
    AddRoundKey(S, K10);
}
  
```

AES-Rijndael Animation: <https://legacy.cryptool.org/de/cto/aes-animation>

Mathematischer Exkurs:

Endliche Körper $\mathbb{F}_{p^n}$

$n=1$: $\mathbb{F}_p := \mathbb{Z}_p$

$n > 1$: $\mathbb{F}_{p^n} := \{a(x) \mid a(x) \in \mathbb{F}_p[x], \text{grad}(a(x)) < n\}$
 $= \{a_{n-1}x^{n-1} + \dots + a_1x + a_0 \mid a_i \in \mathbb{F}_p\}$

+ in $\mathbb{F}_{p^n}$: übliche Polynomaddition
 • in $\mathbb{F}_{p^n}$: s. f. Folie

Polynomdivision mit Rest

Ist $\mathbb{F}$ ein Körper und sind $a(x), b(x) \in \mathbb{F}[x], b(x) \neq 0$, so gibt es eindeutig bestimmte Polynome $q(x), r(x) \in \mathbb{F}[x]$ mit:

$$\underline{a(x)} = \underline{b(x)} \cdot \underline{q(x)} + \underline{r(x)} \quad \text{und} \quad 0 \leq \underline{\text{grad}(r(x))} < \underline{\text{grad}(b(x))} \quad (3.1)$$

Für den *Rest* $r(x)$ der Polynomdivision wird auch

$$a(x) \bmod b(x) := r(x) \quad (3.2)$$

geschrieben.

wil in $\mathbb{Z}$

$\mathbb{F}_{p^n}$ (p **Primzahl**, $n \in \mathbb{N}$, $n > 1$)

Gegeben sei ein irreduzibles Polynom:

$$p(x) \in \mathbb{F}_p[x] \quad \text{mit} \quad \text{grad}(p(x)) = n$$

Dann bilden die Polynome der Menge

$$\mathbb{F}_{p^n} := \{a(x) \mid a(x) \in \mathbb{F}_p[x], \text{grad}(a(x)) < n\}$$

zusammen mit der üblichen Polynomaddition und der Multiplikation

$$a(x) \cdot b(x) := (a(x) \cdot b(x)) \bmod p(x) \quad (3.3)$$

einen Körper. (Das Produkt $a(x) \cdot b(x)$ auf der rechten Seite von (3.3) ist hierbei das übliche Produkt von Polynomen.)

$\leftrightarrow$ entspricht Primzahl n

$\rightarrow$ weitere Annahmen s. Studienheft

Frage 18

Geben Sie für ein Polynom

$$a(x) = a_n x^n + \dots + a_1 x + a_0 \in \mathbb{F}_2[x]$$

$$\overline{\mathbb{F}}_2 = \mathbb{Z}_2$$

Kriterien dafür an, dass

- (i) 0 eine Nullstelle von $a(x)$ ist, $\Leftrightarrow a_0 = 0$
- (ii) 1 eine Nullstelle von $a(x)$ ist, $\Leftrightarrow \sum_{i=0}^n a_i = 0 \quad \Leftrightarrow \text{Anzahl der Koeffizienten} \neq 0 \text{ ist gerade}$

Frage 19

Zeigen Sie, dass Quadrate in $\mathbb{F}_2[x]$ folgendermaßen leicht berechnet werden können:

$$(a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0)^2 = a_n x^{2n} + a_{n-1} x^{2n-2} + \dots + a_1 x^2 + a_0$$

Beispiel 3.1: Irreduzible Polynome über $\mathbb{F}_2$ vom Grad ≤ 4

$$n = \text{Grad}(f)$$

$$n=1: \quad \underline{x}, \quad \underline{x+1}$$

$$n=2: \quad \begin{array}{cccc} x^2, & x^2+1, & x^2+x, & \underline{x^2+x+1} \\ \uparrow & \uparrow & \uparrow & \\ x \cdot x & (x+1)^2 & x(x+1) & \end{array}$$

$$\underline{n=3}: \quad \underline{x^3+x+1}, \quad \underline{x^3+x^2+1} \quad (\text{einzige Polynome ohne Null})$$

$$\underline{n=4}: \quad \underline{x^4+x+1}, \quad \begin{array}{c} x^4+x^2+1 \\ \uparrow \\ (x^2+x+1)^2 \end{array}, \quad \underline{x^4+x^3+1}, \quad \underline{x^4+x^3+x^2+x+1} \quad (-4-)$$

Beispiel 3.2: Irreduzibilität von $m(x) = x^8 + x^4 + x^3 + x + 1 \in \mathbb{F}_2[x]$

1) $m(x)$ besitzt kein Null $\Rightarrow m(x)$ besitzt keinen Linearfaktor

2) $x^8 + x^4 + x^3 + x + 1 : x^2 + x + 1 = x^6 + x^5 + x^3 \quad \text{Rest } x + 1$

$$111000000$$

$$\begin{array}{r}
 m(x): \quad 100011011 \\
 (x^2+x+1) \cdot x^6: \quad 111000000 \\
 \hline
 \quad 11011011 \\
 \quad 11100000 \\
 \hline
 \quad \quad 111011 \\
 \quad \quad 111000 \\
 \hline
 \quad \quad \quad 11 \\
 \quad \quad \quad x+1
 \end{array}$$

Beispiel 3.3: $\mathbb{F}_{256}$, definiert bezüglich $m(x) = x^8 + x^4 + x^3 + x + 1$ 1 0 0 0 | 1 0 1 1

$$x^7 + x^5 + x^4 + x^2 + 1 \leftrightarrow \underline{10110101} \leftrightarrow 0xB5$$

$$x^5 + x^3 + x \leftrightarrow 00101010 \leftrightarrow 0x2A$$

Mit dieser Identifikation entspricht die Addition von zwei Bytewerten ihrer XOR-Verknüpfung:

$$0xB5 + 0x2A \leftrightarrow x^7 + x^4 + x^3 + x^2 + x + 1 \leftrightarrow 10011111 \leftrightarrow 0x9F$$

Die Multiplikation von zwei Bytewerten ist bei dieser Identifikation mittels der Multiplikation der entsprechenden Polynome und der Reduktion des Produktes bezüglich $m(x)$ zu berechnen. Beispielsweise gilt:

$$\underline{0xB5} \cdot \underline{0x2A} \leftrightarrow \underline{(x^7 + x^5 + x^4 + x^2 + 1)(x^5 + x^3 + x)} \bmod m(x)$$

Berechnung des Produkts:

$$\begin{array}{r} \underline{1011010100000} \quad \cdot x^7 \\ \underline{10110101000} \quad \cdot x^3 \\ \underline{101101010} \quad \cdot x \\ \hline 1001001100010 \end{array}$$

$$\begin{array}{r} 1001001100010 \\ 1000110110000 \\ 1000110110000 \\ \hline 1111111111111 \\ \hline 0xFF \end{array}$$

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot \begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix}$$

$$= \begin{pmatrix} 04 \\ . \\ . \\ . \end{pmatrix}$$

$$\begin{array}{r}
 02 \cdot d4 : \quad 110101000 \\
 03 \cdot bf : \quad 10111110 \\
 01 \cdot 5d : \quad 1011101 \\
 01 \cdot 30 : \quad 110000 \\
 \hline
 \quad \quad \quad 100
 \end{array}$$

$$0x04 \quad x^2$$

✓ ■ Kryptographisch sichere Pseudozufallszahlengeneratoren

Cryptographic Secure Pseudo Random Number Generators (CSPRNG)

→ ■ Kryptographische Hashfunktionen

Message Digests (Cryptographic Hash Functions)

✓ ■ Symmetrische Verschlüsselungsverfahren (symmetric ciphers)

■ Message Authentication Codes (MACs)

■ Asymmetrische Verschlüsselungsverfahren (asymmetric ciphers)

■ Digitale Signaturen (digital signatures)

■ Schlüsselaustauschverfahren (key exchange / key establishment)

- DC-02, Kap. 4:

Hashfunktionen

4.1	Grundlagen	54
4.2	MessageDigest-Instanzen: Hashfunktionen in Java- Laufzeitumgebungen	56
4.3	Anwendungsbeispiele	58
4.4	Brute-Force-Angriffe auf Hashfunktionen	63
4.5	Konstruktionsverfahren von Hashfunktionen	69



- *Werte einer Hashfunktion werden auch als **digitale Fingerabdrücke (Digital Fingerprints)** bezeichnet.*

Gegeben sei eine Nachrichtenmenge $\mathcal{M}$, beispielsweise:

$$\mathcal{M} = \mathcal{Z}^{\geq 0} := \bigcup_{n \in \mathbb{N}_0} \mathcal{Z}^n = \{(z_1, \dots, z_n) \mid z_i \in \mathcal{Z}, n \in \mathbb{N}_0\} \quad \text{mit } \mathcal{Z} = \mathbb{Z}_2$$

Eine Abbildung

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

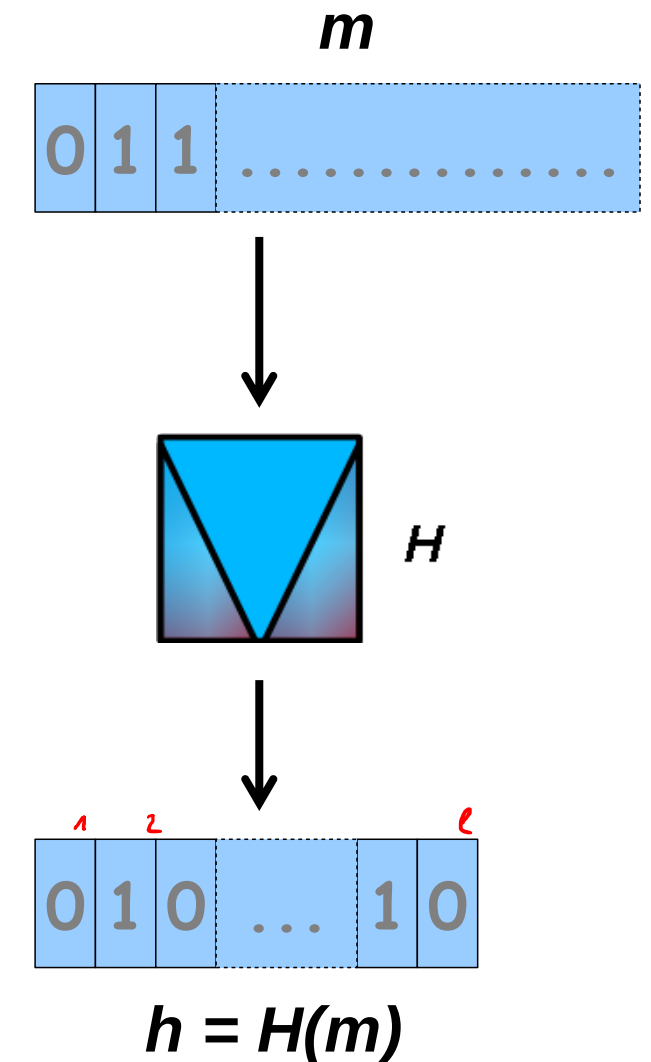
mit einem festen $l \in \mathbb{N}$ wird *Hashfunktion* genannt, falls folgende Bedingungen gelten:

(i) Einwegfunktion (Preimage resistance)

Sei $h = (h_1, h_2, \dots, h_l) \in (\mathbb{F}_2)^l$ gegeben.

Dann ist es *praktisch unmöglich* eine Nachricht

$m = (z_1, z_2, z_3, \dots, z_n)$ zu finden, für die gilt: $H(m) = h$.



Gegeben sei eine Nachrichtenmenge $\mathcal{M}$, beispielsweise:

$$\mathcal{M} = \mathcal{Z}^{\geq 0} := \bigcup_{n \in \mathbb{N}_0} \mathcal{Z}^n = \{(z_1, \dots, z_n) \mid z_i \in \mathcal{Z}, n \in \mathbb{N}_0\} \quad \text{mit } \mathcal{Z} = \mathbb{Z}_2$$

Eine Abbildung

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

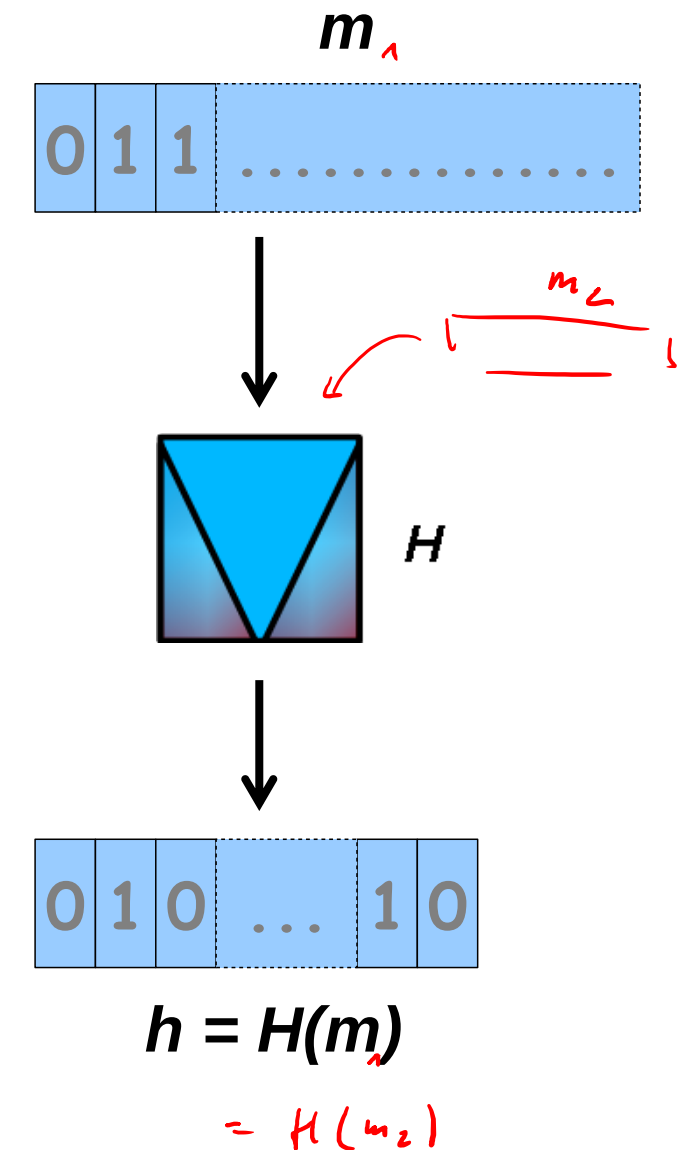
mit einem festen $l \in \mathbb{N}$ wird *Hashfunktion* genannt, falls folgende Bedingungen gelten:

(ii) Kollisionsresistenz (Collision resistance)

Es ist es *praktisch unmöglich* zwei verschiedene

Nachrichten $m_1 = (z_1, z_2, \dots, z_{n_1})$ und $m_2 = (z'_1, z'_2, \dots, z'_{n_1})$

zu finden, für die gilt: $H(m_1) = H(m_2)$



Gegeben sei eine Nachrichtenmenge $\mathcal{M}$, beispielsweise:

$$\mathcal{M} = \mathcal{Z}^{\geq 0} := \bigcup_{n \in \mathbb{N}_0} \mathcal{Z}^n = \{(z_1, \dots, z_n) \mid z_i \in \mathcal{Z}, n \in \mathbb{N}_0\} \quad \text{mit } \mathcal{Z} = \mathbb{Z}_2$$

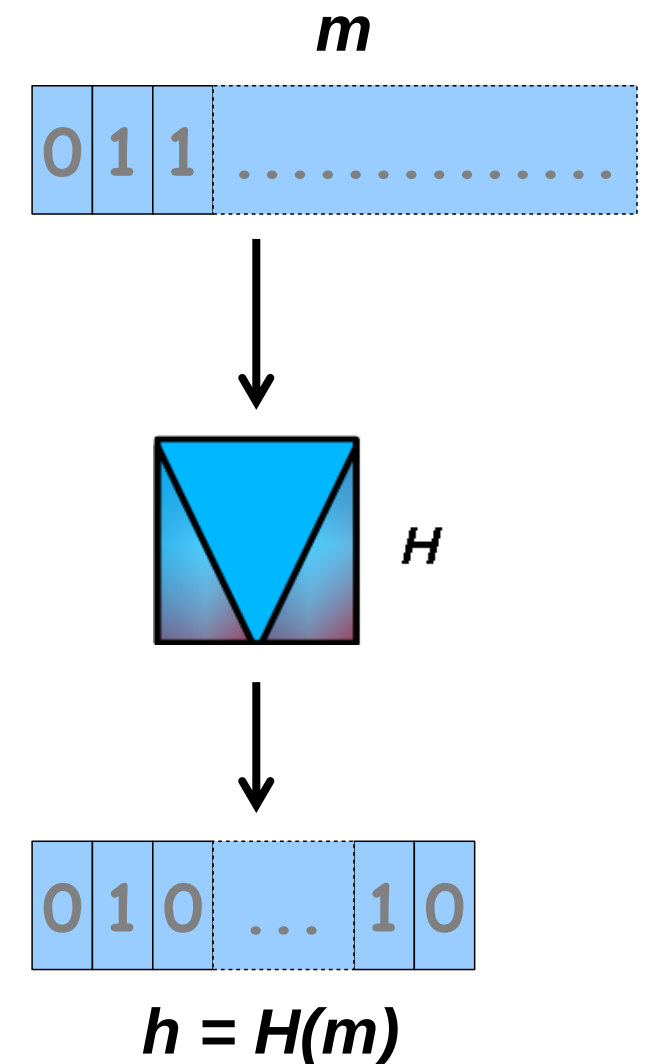
Eine Abbildung

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

mit einem festen $l \in \mathbb{N}$ wird *Hashfunktion* genannt, falls folgende Bedingungen gelten:

- (i) H ist eine *Einwegfunktion*, d. h. es gibt zu keinem $y \in \mathbb{Z}_2^l$ einen effizienten Algorithmus zum Auffinden eines Urbilds (d. h. einer Nachricht) m mit $H(m) = y$.
Diese Eigenschaft wird auch als *Urbildresistenz* (*preimage resistance*) bezeichnet.
- (ii) H ist (*stark*) *kollisionsresistent*, d. h. es gibt keinen effizienten Algorithmus, um eine *Kollision* zu finden, wobei eine Kollision durch zwei Nachrichten m_1, m_2 mit $m_1 \neq m_2$ und $H(m_1) = H(m_2)$ definiert ist.

Unter einem *effizienten Algorithmus* wird ein Algorithmus verstanden, der das Auffinden von Urbildern bzw. von Kollisionen auf eine effizientere Art als durch eine Brute-Force-Suche ermöglicht.



Kommentar Die beiden Eigenschaften (i) und (ii) einer Hashfunktion sind tatsächlich unabhängige Eigenschaften, d.h. weder ist eine Einwegfunktion notwendigerweise kollisionsresistent noch folgt aus der Kollisionsresistenz einer Funktion, dass diese eine Einwegfunktion sein muss. Hierzu die folgenden Beispiele:

- Es sei $\mathcal{M} = \mathbb{Z}_2^{\geq 0}$ und $H : \mathcal{M} \rightarrow \{0, 1\}^l$ eine Hashfunktion. Dann wird durch

$$\tilde{H} : \mathcal{M} \rightarrow \{0, 1\}^l, \quad \tilde{H}((m_1, \dots, m_n)) = \begin{cases} H(()) & \text{für } n = 0 \\ H((m_2, \dots, m_n)) & \text{sonst} \end{cases}$$

eine Einwegfunktion definiert, die aber nicht mehr kollisionsresistent ist.

- Es sei wiederum $\mathcal{M} = \mathbb{Z}_2^{\geq 0}$ und $H : \mathcal{M} \rightarrow \{0, 1\}^l$ eine Hashfunktion. Durch

$$\hat{H} : \mathcal{M} \rightarrow \{0, 1\}^{l+1}$$

$$\hat{H}((m_1, \dots, m_n)) = \begin{cases} (0, m_1, \dots, m_l) & \text{für } n = l \\ (1, h_1, \dots, h_l) & \text{mit } (h_1, \dots, h_l) = H((m_1, \dots, m_n)) \text{ für } n \neq l \end{cases}$$

wird dann eine Funktion definiert, die nach wie vor kollisionsresistent ist.

$\hat{H}$ ist aber keine Einwegfunktion mehr, da für die Hälfte aller möglichen Bilder der Funktion $\hat{H}$ (nämlich alle Bitfolgen der Länge $l + 1$, die mit 0 beginnen) sofort ein Urbild angegeben werden kann. (Der entsprechende Algorithmus ist trivialerweise effizient.)

Schwache Kollisionsresistenz (Second preimage resistance)

Eine Abbildung

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

heißt *schwach-kollisionsresistent*, falls es keinen effizienten Algorithmus gibt um zu einem beliebigen $m \in \mathcal{M}$ ein $\tilde{m} \in \mathcal{M}$ mit

$$\tilde{m} \neq m \quad \text{und} \quad H(\tilde{m}) = H(m)$$

zu finden.

Ein Angriff auf die schwache Kollisionsresistenz ist mindestens so aufwendig, wie ein Angriff auf die Kollisionsresistenz.

Ist die Bestimmung einer Kollision praktisch nicht möglich, so kann auch die schwache Kollisionsresistenz in der Praxis nicht erfolgreich angegriffen werden.

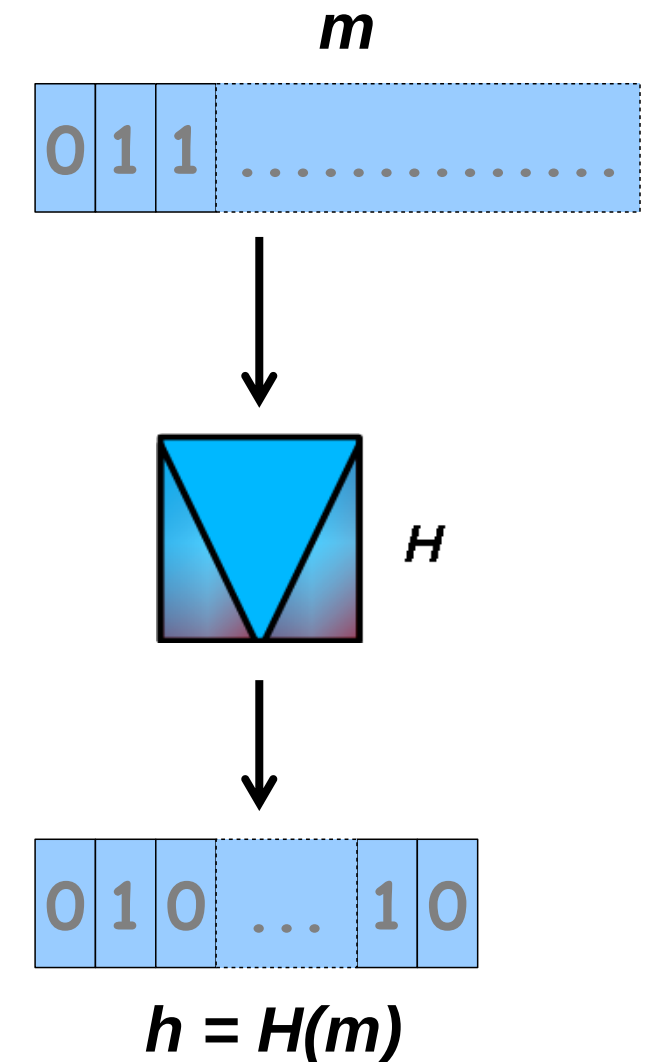
Kommentar Üblicherweise werden Hashfunktionen so konstruiert, dass gilt:

- Die Berechnung von Hashwerten ist effizient möglich und die Berechnungszeit ist (bei längeren Nachrichten) im Wesentlichen proportional zur Nachrichtenlänge.
- $\mathcal{M}$ ist eine Teilmenge von $\mathcal{Z}^{\geq 0}$ mit $\mathcal{Z} = \mathbb{Z}_2^8$, wobei die maximale Länge der in $\mathcal{M}$ vorkommenden Bytefolgen durch eine sehr große Zahl $N \in \mathbb{N}$ beschränkt ist:

$$\mathcal{M} = (\mathbb{Z}_2^8)^{\leq N} := \{(z_1, \dots, z_n) \mid z_i \in \mathbb{Z}_2^8, n \in \mathbb{N}_0, n \leq N\}$$

In der Regel ist N so groß, dass alle in einer Anwendung vorkommenden Nachrichten eine kleinere Länge besitzen.

- Die Bitlänge l der Hashwerte ist ein Vielfaches von 8, sodass Hashwerte auch als $\frac{l}{8}$ -Tupel von Bytewerten aufgefasst werden können.



- FIPS PUB 180-4: Secure Hash Standard (SHS) (2015)

- SHA-1 (FIPS 180, 1993)

$l = 160$

zuw. Länge: MD5

- SHA-256, SHA-384, SHA-512 (FIPS PUB 180-2, 2002)

- SHA-224 (FIPS PUB 180-3, 2008)

- SHA-512/224, SHA-512/256

- FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions (2015)

- SHA3-224, SHA3-256, SHA3-384, SHA3-512

- SHAKE128 and SHAKE256 (extendable-output functions (XOFs))

■ Security Strengths of Approved Hash Functions

The security strengths of NIST approved hash functions are summarized below. $L(M)$ is defined as $\lceil L(M) = \log_2 \frac{\text{len}(M)}{B} \rceil$, where $\text{len}(M)$ is the length of message M in bits and B is the block length of the functions in bits, $B = 512$ for SHA-1, SHA-224, and SHA-256 and $B = 1024$ for SHA-384, SHA-512, SHA-512/224, and SHA-512/256.

	Collision Resistance Strength in bits	Preimage Resistance Strength in bits	Second Preimage Resistance Strength in bits
SHA-1	<80	160	$160 - L(M)$
SHA-224	112	224	$\min(224, 256 - L(M))$
SHA-256	128	256	$256 - L(M)$
SHA-384	192	384	384
SHA-512	256	512	$512 - L(M)$
SHA-512/224	112	224	224
SHA-512/256	128	256	256
SHA3-224	112	224	224
SHA3-256	128	256	256
SHA3-384	192	384	384
SHA3-512	256	512	512

```
1  InputStream is = new FileInputStream("rfc4270.txt");  
   byte[] m = is.readAllBytes();  
  
   MessageDigest md = MessageDigest.getInstance("SHA-256");  
5  byte[] hashValue = md.digest(m);  
   System.out.println("Hashwert:");  
   System.out.println(Dump.dump(hashValue));
```


Beispiel 4.1: Kollision für MD5

Im Jahr 2004 wurden von Xiaoyun Wang, Dengguo Feng, Xuejia Lai und Hongbo Yu in der Arbeit *Collisions for Hash Functions MD4, MD5, HAVAL-128 and RIPEMD* (<http://eprint.iacr.org/2004/199>) erstmals Kollisionen für die Hashfunktion MD5 publiziert.

```
1  byte[] data1 = Dump.hexString2byteArray(""
   + "D131DD02C5E6EEC4693D9A0698AFF95C2FCAB58712467EAB"
   + "4004583EB8FB7F8955AD340609F4B30283E488832571415A"
   + "085125E8F7CDC99FD91DBDF280373C5BD8823E3156348F5B"
5  + "AE6DACD436C919C6DD53E2B487DA03FD02396306D248CDA0"
   + "E99F33420F577EE8CE54B67080A80D1EC69821BCB6A88393"
   + "96F9652B6FF72A70");

10 byte[] data2 = Dump.hexString2byteArray(""
   + "D131DD02C5E6EEC4693D9A0698AFF95C2FCAB50712467EAB"
   + "4004583EB8FB7F8955AD340609F4B30283E4888325F1415A"
   + "085125E8F7CDC99FD91DBD7280373C5BD8823E3156348F5B"
   + "AE6DACD436C919C6DD53E23487DA03FD02396306D248CDA0"
   + "E99F33420F577EE8CE54B67080280D1EC69821BCB6A88393"
15 + "96F965AB6FF72A70");
```

Frage 23

Berechnen Sie die MD5-Hashwerte von data1 und data2 aus Abbildung 4.2 und überzeugen Sie sich davon, dass sie gleich sind.

lies-Text

An wie vielen Bitstellen unterscheiden sich data1 und data2?

Beispiel 4.2: Kollision für SHA-1

Seit dem Jahr 2005 sind diverse Arbeiten publiziert worden, die zeigen, dass ein Auffinden von Kollisionen für die Hashfunktion SHA-1 deutlich effizienter als mit einem Brute-Force-Angriff möglich ist. Dennoch dauerte es ungefähr weitere 12 Jahre bis im Jahre 2017 eine SHA-1-Kollision erstmals öffentlich publiziert wurde.

Unter <https://shattered.io> können Sie Links sowohl zu dieser Publikation, die in der Einführung eine lesenswerte historische Zusammenfassung der zuvor bereits publizierten Angriffe auf die Hashfunktionen der MD-SHA-Familie und deren praxisrelevanten Implikationen enthält, als auch zu zwei verschiedenen PDF-Dateien shattered-1.pdf und shattered-2.pdf finden.

veraltet

Frage 24

Betrachten Sie die in Beispiel 4.2 genannten Dateien shattered-1.pdf und shattered-2.pdf und überzeugen Sie sich so von deren Verschiedenheit.

Berechnen Sie die SHA-1-Hashwerte von shattered-1.pdf und shattered-2.pdf und überzeugen Sie sich von deren Gleichheit.

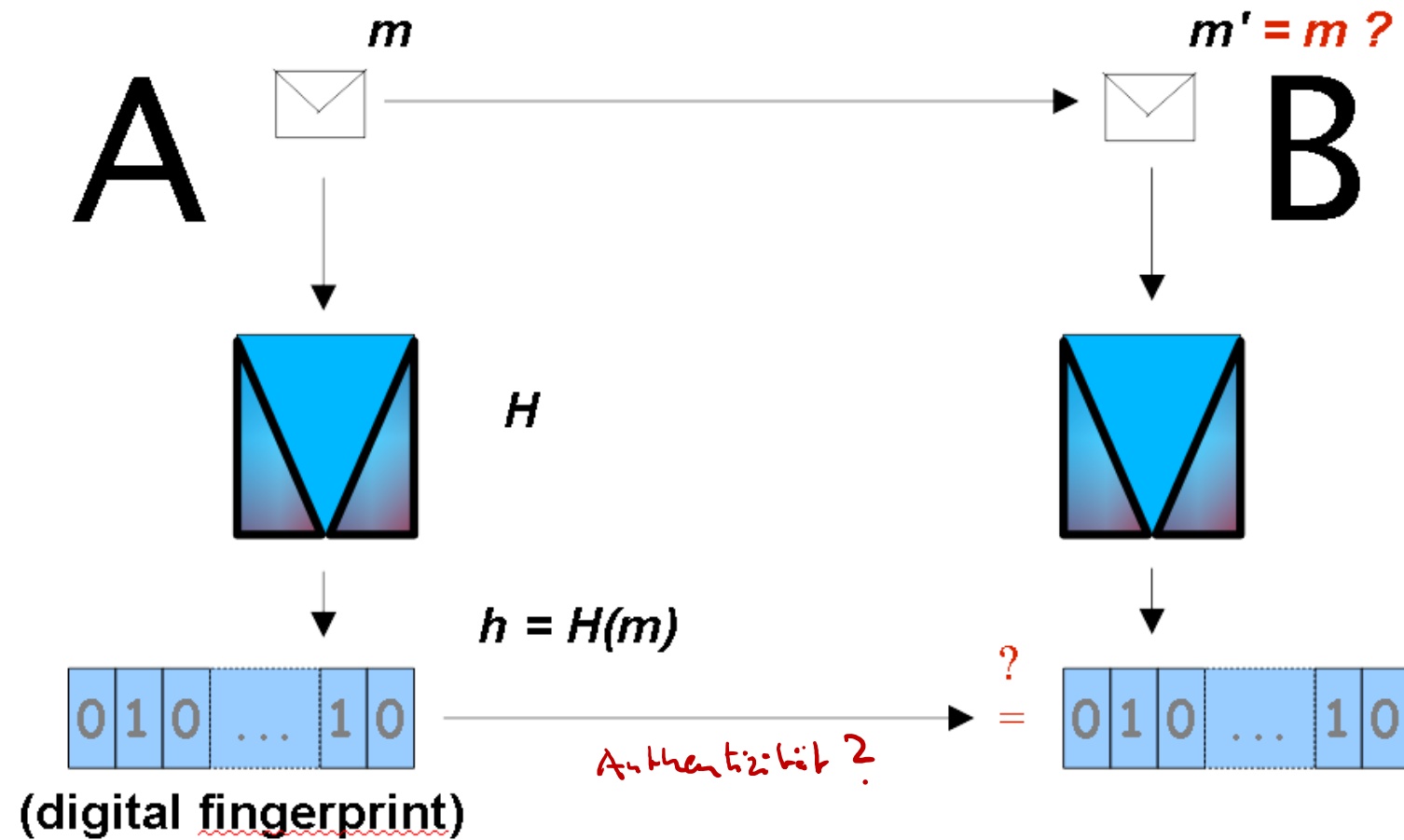
An wie vielen Bitstellen unterscheiden sich shattered-1.pdf und shattered-2.pdf? In welchem Bereich der Dateien (Bytestellen) befinden sich die Unterschiede?

klein - Test

- Funktionsbausteine zur Konstruktion von:
 - CSPRNGs (Beispiel: Hash_DRBG)
 - MACs (HMAC)
- Integritätscheck von:
 - übertragenen Daten (Beispiel: Hashwertüberprüfung nach Dateidownload)
 - gespeicherten Daten (Beispiel: sicheres OS mit Whitelist erlaubter ausführbarer Dateien)
- Schutz geheimer Daten, die einer Identifikation dienen (Beispiel: Passwortdateien)
- Password Based Key Derivation Functions
 - PBKDFs aus PKCS #5 (Ref.: [RFC 8018](#), s.a. [NIST SP 800-132](#))
 - PBKDF1 (entspricht dem Anwendungsbeispiel: Passwortdateien mit Salt und Iteration Count)
 - PBKDF2 (basiert auf HMACs)

Anwendungsbeispiel: Integritätsschutz bei einem Dateidownload

Häufig wird auf Webseiten neben einem Link zum Download einer Datei auch ein Hashwert der Datei angegeben. Nach einem Download der Datei kann der Hashwert berechnet und mit dem auf der Webseite angegebenen verglichen werden.



Anwendungsbeispiel: Integritätsschutz bei einem Dateidownload

Häufig wird auf Webseiten neben einem Link zum Download einer Datei auch ein Hashwert der Datei angegeben. Nach einem Download der Datei kann der Hashwert berechnet und mit dem auf der Webseite angegebenen verglichen werden.

Diese Überprüfung bietet aber so gut wie keinen Schutz gegen einen Angreifer, der eine solche Webseite erfolgreich angreift und die Datei durch eine Datei mit Schadsoftware ersetzt. Der Angreifer wird in der Regel auch ohne Probleme die Angabe des Hashwerts auf der angegriffenen Webseite geeignet anpassen können.

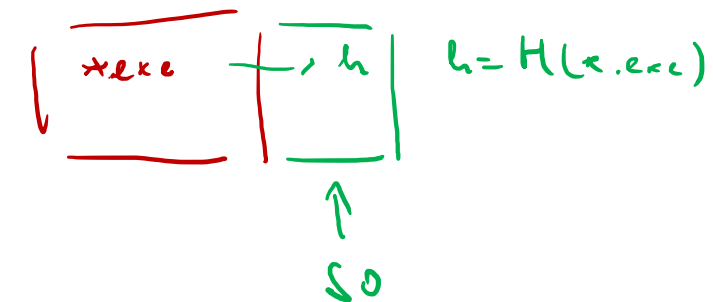
Ein gewisser Schutz vor dem Download von Schadsoftware ist möglich, wenn die entsprechende Datei auch auf anderen Webseiten zum Download zur Verfügung steht. Nach dem Download der Datei und der Berechnung ihres Hashwerts kann dieser dann nämlich mit der Angabe von einer anderen Webseite verglichen werden. ◀

Anwendungsbeispiel: Integritätsschutz von Dateien

Soll in einem Rechnersystem sichergestellt werden, dass nur solche Dateien zur Ausführung kommen, denen aufgrund definierter Kriterien vertraut wird, so lässt sich dies durch eine Erweiterung des Betriebssystems folgendermaßen erreichen:

Zunächst werden mit einer Hashfunktion die Hashwerte der zulässigen ausführbaren Dateien berechnet und in einer Tabelle (Whitelist) in einem sicheren Speicherbereich, d. h. einem Speicherbereich auf den ein potentieller Angriff keinen Zugriff hat, abgelegt. Soll eine Datei zur Ausführung gebracht werden, so wird nun vom Betriebssystem der Hashwert der Datei berechnet und mit dem sicher abgespeicherten verglichen. Sind die Werte verschieden, so wurde die Datei offensichtlich verändert und ihre Ausführung wird deshalb unterbunden. (Dateien, für die kein Eintrag in der Whitelist enthalten ist, werden natürlich auch nicht ausgeführt.)

Versucht ein Angreifer eine in der Whitelist aufgeführte Datei mit Schadcode abzuändern, so sieht er sich vor die Herausforderung gestellt, dass die modifizierte Datei den gleichen Hashwert besitzen muss wie die ursprüngliche Datei. Ist die Hashfunktion schwach kollisionsresistent, so sollte ihm dies nicht gelingen!



Anwendungsbeispiel: Integritätsschutz von Dateien

Soll in einem Rechnersystem sichergestellt werden, dass nur solche Dateien zur Ausführung kommen, denen aufgrund definierter Kriterien vertraut wird, so lässt sich dies durch eine Erweiterung des Betriebssystems folgendermaßen erreichen:

Zunächst werden mit einer Hashfunktion die Hashwerte der zulässigen ausführbaren Dateien berechnet und in einer Tabelle (*Whitelist*) in einem sicheren Speicherbereich, d. h. einem Speicherbereich auf den ein potentieller Angriff keinen Zugriff hat, abgelegt. Soll eine Datei zur Ausführung gebracht werden, so wird nun vom Betriebssystem der Hashwert der Datei berechnet und mit dem sicher abgespeicherten verglichen. Sind die Werte verschieden, so wurde die Datei offensichtlich verändert und ihre Ausführung wird deshalb unterbunden. (Dateien, für die kein Eintrag in der Whitelist enthalten ist, werden natürlich auch nicht ausgeführt.)

Eine etwas andere Situation liegt vor, falls die Entwicklerin einer zulässigen ausführbaren Datei böse Absichten hegt. Sie kann nämlich im Voraus versuchen zwei Dateien mit unterschiedlicher Funktionalität (z. B. ohne und mit einer *Backdoor*) zu erstellen, die den gleichen Hashwert besitzen. Ist die Hashfunktion kollisionsresistent, so sollte ihr dies aber nicht gelingen. Die schwache Kollisionsresistenz begründet in diesem Fall jedoch noch keinen Schutz vor einem solchen Angriff. ◀

Anwendungsbeispiel: Passwortdateien

u_i	p_i
Alice	123456
Bob	rj5gt7hog4m9gxtinokx
Charly	admin
$\vdots$	$\vdots$

Abb. 4.4 Passwortdatei

u_i	$H(p_i)$
Alice	8D969E...DC6C92
Bob	714A52...D2D382
Charly	8C6976...48A918
$\vdots$	$\vdots$

Abb. 4.5 Passwortdatei mit den SHA-256-Hashwerten der Passwörter aus Abbildung 4.4

Anwendungsbeispiel: Passwortdateien mit Salt und Iteration Count

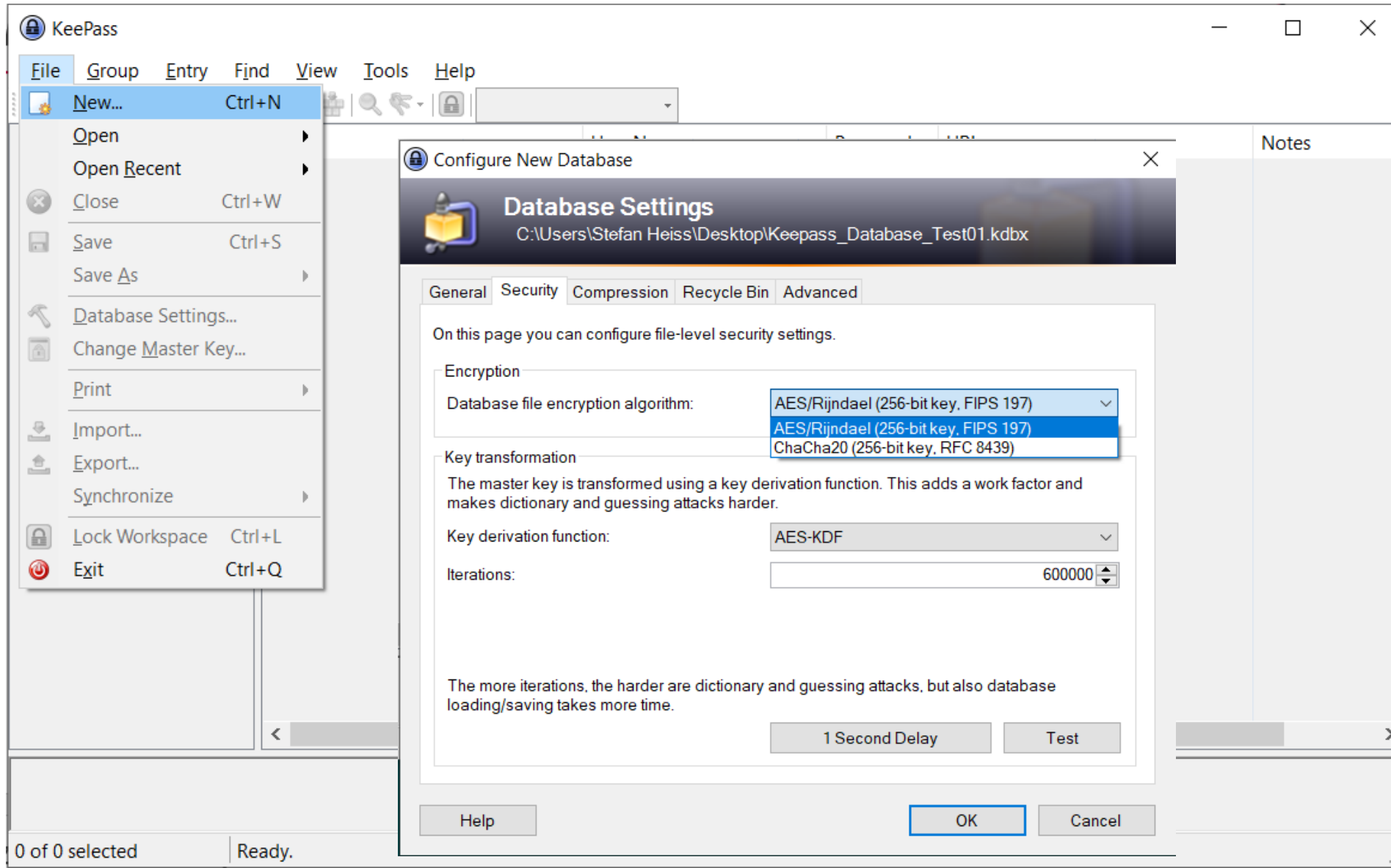
Zur Abwehr möglicher Wörterbuchangriffe wird jedem Nutzer u_i ein zufällig gewählter sogenannter *Salt*-Wert zugeordnet. Dieser wird zusammen mit dem Nutzernamen in der Passwortdatei gespeichert. Das Passwort p_i des Nutzers u_i wird nun vor der Hashberechnung um den Salt-Wert s_i ergänzt, d. h. anstelle von $H(p_i)$ wird $H(p_i||s_i)$ berechnet und dem Eintrag für den Nutzer u_i in der Passwortdatei hinzugefügt.

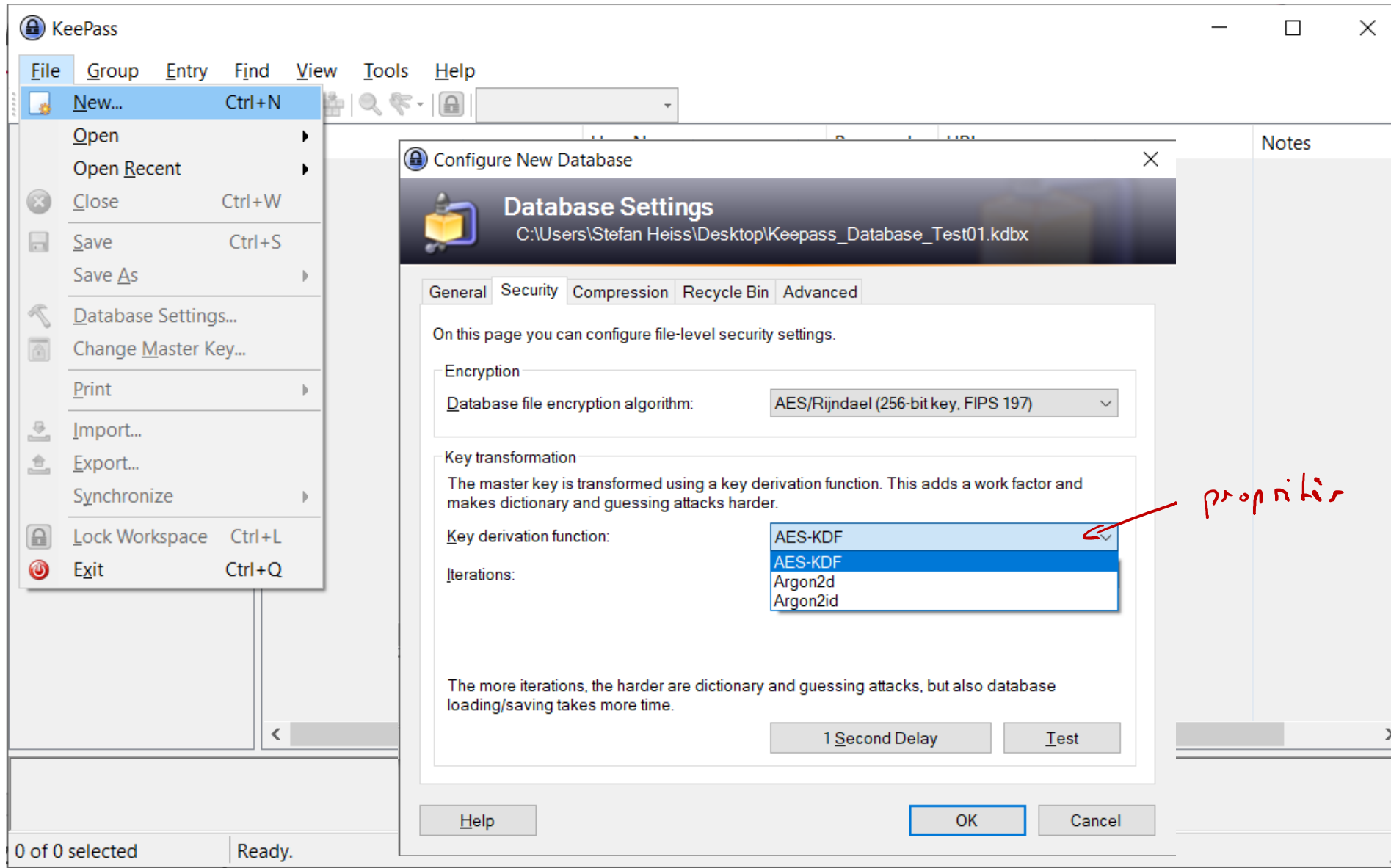
Schließlich kann der Aufwand für nutzerspezifische Brute-Force-Angriffe noch dadurch deutlich erhöht werden, indem nicht nur eine sondern eine größere Anzahl c (z. B. $c = 10^6$) von Hashberechnungen zur Berechnung der Einträge in der Passwortdatei erforderlich sind. Hierzu wird für den Nutzer u_i anstelle von $H(p_i||s_i)$ beispielsweise der Wert

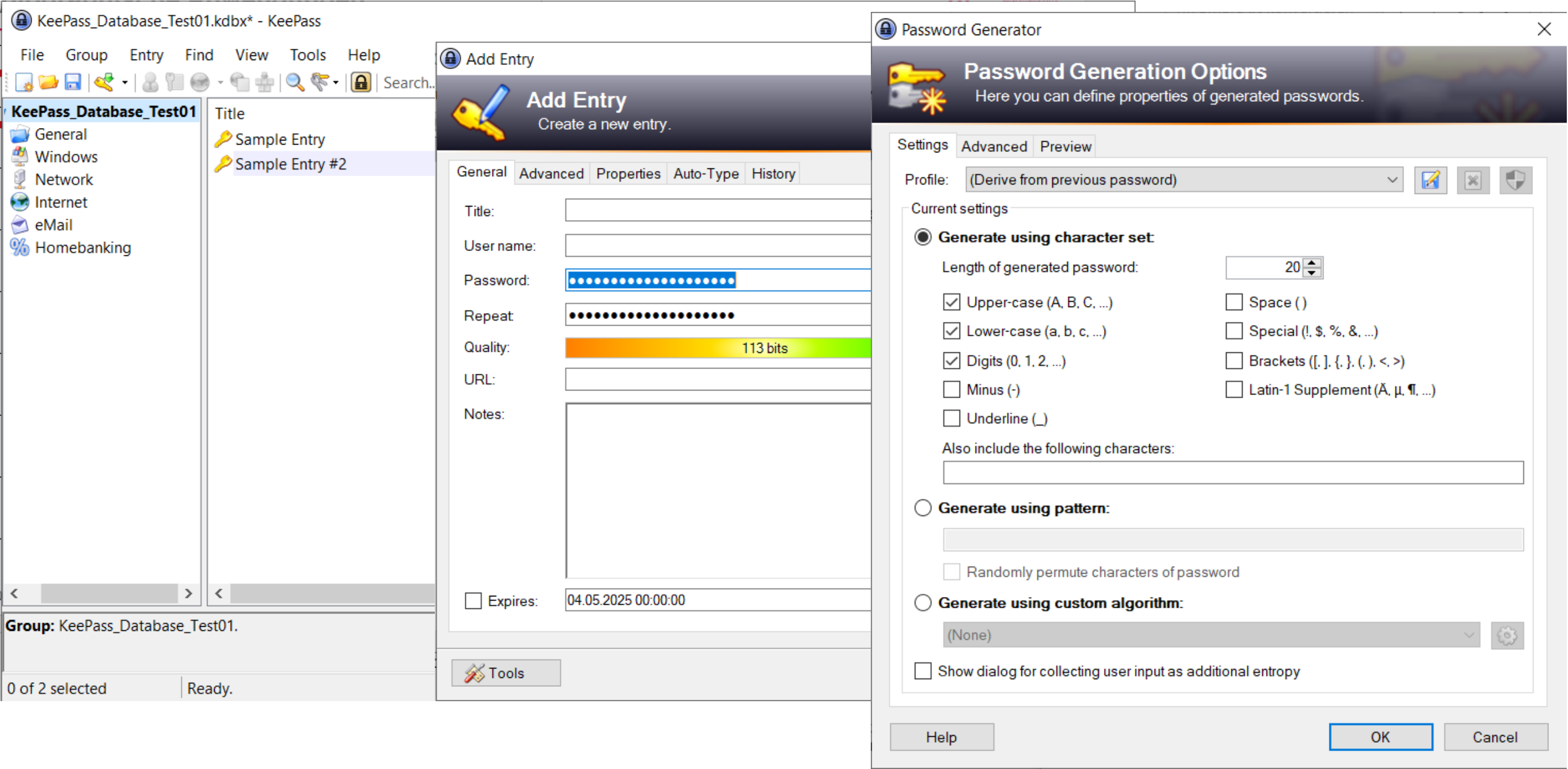
$$H^c(p_i||s_i) = \underbrace{H(H(\dots H(p_i||s_i)\dots))}_c \quad (*)$$

in der Passwortdatei gespeichert. Die Zahl c wird als *Iteration Count* bezeichnet.

u_i	s_i	$H^c(p_i s_i)$
Alice	22445CBCBDFD10E00EA6	DADCC5...786532
Bob	6E25B70D10ABAC317C54	6FDAD4...87FD6E
Charly	83CCA240B57595D58192	2C2FA8...68B999
:	:	:







- [NIST SP 800-63B-4 \(2nd Public Draft\)](#)

- **Digital Identity Guidelines:**

Authentication and Authenticator Management

- [3.1.1.2. Password Verifiers](#)
- [Appendix A. Strength of Passwords](#)

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.SP.800-63b-4.2pd>

August 2024



U.S. Department of Commerce
Gina M. Raimondo, Secretary

National Institute of Standards and Technology

Verifiers and CSPs ...

1. ... SHALL require passwords to be a minimum of eight characters in length and SHOULD require passwords to be a minimum of 15 characters in length.
2. ... SHOULD permit a maximum password length of at least 64 characters.
3. ... SHOULD accept all printing ASCII characters and the space character in passwords.
4. ... SHOULD accept Unicode characters in passwords. Each Unicode code point SHALL be counted as a single character when evaluating password length.
5. ... SHALL NOT impose other composition rules (e.g., requiring mixtures of different character types) for passwords.
6. ... SHALL NOT require users to change passwords periodically. ...
7. ... SHALL NOT permit the subscriber to store a hint that is accessible to an unauthenticated claimant.
8. ... Verifiers and CSPs SHALL NOT prompt subscribers to use knowledge-based authentication (KBA) (e.g., “What was the name of your first pet?”) or security questions when choosing passwords.
9. Verifiers SHALL verify the entire submitted password (i.e., not truncate it).